CS60075 · NLP ASSIGNMENT II · PROJECT 4

Extractive and abstractive summarization on XSum

Deepanshu Olakh | 24CS10042 | Individual submission

November 2026

This study compares two sentence rankers, a trained pointer-generator with coverage, and a pipeline that supplies the same generator with ranked sentences. The research question is whether narrowing the input to salient sentences changes reference overlap and source support on a corpus whose reference summaries are highly abstractive.

Experimental design

The experiment used 16,376 official training articles, 240 validation articles, and 600 test articles after filtering. All methods were evaluated on the same ordered test IDs. The neural systems share a single trained checkpoint, so the direct comparison changes the text presented at inference.

Main measurements

System	ROUGE-1	ROUGE-2	ROUGE-L
TextRank	19.11	2.91	13.82
LexRank	18.94	3.02	13.88
Pointer-generator	18.19	2.69	15.39
Pipeline	17.63	2.50	14.97

The pipeline changed ROUGE-L by -0.42 points relative to the full-article pointer-generator. Content-word overlap with the selected extract changed by +21.64 points. These are descriptive differences on the held-out subset.

Official XSum splits

XSum pairs BBC articles with one-sentence summaries written as news introductions [1]. The reference often compresses or recasts the main event, so selecting a faithful source sentence can still produce low reference overlap. The article identifier is used only to check that methods were scored on the same examples.

Split	Usable examples	Use
Train	16,376	IDF, vocabulary, neural weights
Validation	240	Graph and pipeline selection
Test	600	Final comparison

Sampling and preprocessing

The loader reads the official Parquet splits, uses NumPy PCG64 with seed 24 to sample rows uniformly without replacement within each split, restores the official row order and filters entries with no usable source or reference tokens. Selection occurs before filtering. All validation choices are made within the validation split. The code checks the ordered test IDs before joining saved predictions.

The graph preprocessing splits articles into newline-delimited paragraphs and applies NLTK Punkt within each paragraph. Common titles and abbreviations are supplied explicitly so the segmenter needs no external model download. Rare abbreviations can still be segmented incorrectly. XSum references are used as supplied. The generator lowercases and tokenizes words and numbers, retains common contractions, and discards most punctuation.

The graph rankers see the entire article. The full-article pointer-generator receives the leading tokens up to its source cap, and the hybrid receives the leading tokens of the selected extract up to the same cap. This distinction matters when interpreting the effect of selecting sentences beyond the article prefix.

TextRank and LexRank

The implementation fits a TF-IDF vocabulary of up to 50,000 features on training sentences. It transforms the sentences of each article into unit-normalized vectors and computes their pairwise cosine similarities. The graph is undirected and self-loops are removed.

Both systems compute weighted PageRank with damping 0.85 and uniform teleportation. A sentence with no outgoing weight distributes its mass uniformly. Iteration stops at an L1 change below 1e-9 or after 100 updates. TextRank retains all positive edges [2]. LexRank removes weights below a threshold before ranking [3]. Ties preserve sentence order.

Validation choices

The sweep tries k = 1, 2, 3 for both systems and cosine thresholds 0.1, 0.2, 0.3 for LexRank. The objective is mean validation ROUGE-L F1. After selection, the original sentence order is restored in the extract.

System	Threshold	k	Validation R-L
TextRank	0.0	1	14.31
LexRank	0.1	1	13.76

TextRank is carried forward with k = 1. This choice uses validation performance. Short extracts match the one-sentence reference format more closely, while longer extracts can raise recall at the cost of precision. Edge pruning can separate weakly related topics, and a high threshold can also leave several sentences disconnected.

03 / TASK 2

Pointer-generator with coverage

A bidirectional LSTM encodes the source. The final forward and backward hidden and cell states initialize the decoder through learned projections. The LSTM decoder receives the previous token embedding and the previous attention context. The encoder skips padded positions and additive Bahdanau attention masks padding before normalizing over the source.

At each decoding step, a sigmoid gate combines a vocabulary softmax with the attention distribution. The copy branch scatters probability into document-specific extended vocabulary IDs and sums contributions from repeated source words. A reference word outside the fixed vocabulary can receive a copy ID if it occurs in the truncated source. A copied OOV token is fed back to the next step as UNK.

Coverage and training

Coverage is the sum of past attention at each source position. It enters the attention energy and penalizes overlap between the current attention and prior coverage. The loss is the mean over non-padding target tokens of negative log likelihood plus the coverage penalty with weight 1.0. Training uses teacher forcing, Adagrad at 0.15 with initial accumulator 0.1, and gradient norm clipping at 2.

Setting	measured run	Assignment target
Embedding / LSTM width	48 / 48	128 / 256
Fixed vocabulary	6,000	Up to 50,000
Source / target cap	160 / 28	Reduced for this run
Beam width / batch size	4 / 32	Beam width 4

Beam search blocks PAD, BOS, and UNK outputs, permits EOS after four generated tokens, and ranks candidates with length-normalized log probability. The output is detokenized as space-separated lowercase words. The code saves an optimizer and model checkpoint after each completed epoch.

04 / TASK 3

Extract then abstract

The hybrid ranks the full article with TextRank, using the same validation-selected graph settings as Task 1. It restores the original order of the top m sentences and runs the Task 2 checkpoint over those sentences. The pipeline reuses the same vocabulary, decoder, and source cap.

m	Validation ROUGE-L
2	15.27
4	15.33
6	15.99

The selected m was 6, above the Task 1 value of k = 1. An article with fewer than m sentence units contributes all available units. This pipeline may expose information beyond the prefix seen by the full-article model. It can also discard context such as attribution and antecedents.

Measuring source overlap

For each test article, all systems are scored against the same selected extract for the factual-consistency proxy. The numerator counts output content-word occurrences present in that extract. The denominator counts all output content-word occurrences. Lowercase word tokens are filtered with the scikit-learn English stopword list, and a prediction with no content words receives zero.

An extract can share every entity word with a prediction while the prediction assigns those entities the wrong roles. The proxy therefore measures lexical support. A high score should be interpreted alongside the article and reference. Using the same extract across systems makes the measure comparable, while the hybrid has direct access to that evidence during decoding.

Matched test comparison

System	R-1	R-2	R-L	Comp.	Density	Proxy
TextRank	19.11	2.91	13.82	0.105	23.88	100.0%
LexRank	18.94	3.02	13.88	0.100	22.73	99.1%
Pointer-generator	18.19	2.69	15.39	0.069	1.58	75.5%
Pipeline	17.63	2.50	14.97	0.069	1.60	97.2%

ROUGE reports mean per-example F1 on a 0–100 scale. ROUGE-1 and ROUGE-2 use unigram and bigram overlap. ROUGE-L uses the longest common subsequence. The scoring library applies stemming. Compression is summary word count divided by full source word count. Density sums squared lengths of greedily matched source fragments and divides by summary word count.

Pointer-generator had the highest mean test ROUGE-L (15.39). Relative to the full-article pointer-generator, restricting the input changed ROUGE-L by -0.42 points and selected-extract overlap by +21.64 points. The proxy rises while ROUGE-L falls. The lexical-overlap metric is aligned with the hybrid input, so the proxy difference cannot establish a corresponding change in factual accuracy.

Training and comparison with See et al.

Epoch	Token loss	Seconds
1	7.161	60.5
2	6.809	60.1
3	6.605	60.7

Our pointer-generator reached 18.19 / 2.69 / 15.39 ROUGE-1/2/L. See et al. report 39.53 / 17.28 / 36.38 with coverage on CNN/Daily Mail [4]. The respective differences are -21.34 / -14.59 / -20.99 points. Their corpus, reference length, ROUGE implementation, model size, and training schedule differ. They added coverage after training the pointer-generator, while this measured run trained with coverage from the first update.

Generated summaries

Partial success in both systems · BBC ID 39531985

Reference — TV presenter Paul O'Grady has been invited on a tour of Southend-on-Sea after reportedly lambasting it during filming of the programme Blind Date.

Full article — a channel o'grady has been a channel of the channel of the channel of the channel

Selected sentences — a channel o'grady has been a channel of the bbc of the bbc of the southend

Both versions retain the presenter's surname. The hybrid also recovers Southend from the selected sentences. Both miss the invitation and repeat outlet-related words. Entity overlap earns ROUGE credit even though the main event is missing.

The hybrid recovers a technical detail · BBC ID 35695680

Reference — The refurbishment of the first of 13 aircraft operated by the Scottish airline Loganair has been completed.

Full article — a airline's has been to the public to the public of the public of the airline's

Selected sentences — the renewal has been a project of the saab has been

The full-article output stays generic and repeats “the public.” The hybrid mentions renewal and Saab, which is relevant to the aircraft story. It leaves out Loganair and the completion of the first refurbishment, so the improvement is partial.

Lexical support without a useful summary · BBC ID 36135425

Reference — Euro 2016 will be contested by 24 teams over 30 days at 10 different venues in France.

Full article — the ireland of the england of the england of the england of the england in the england

Selected sentences — the stade has been the england of the stade of the stade of the stade

The hybrid repeats venue and team words and omits the number of teams, days and venues. It scores 100% on the selected-extract overlap proxy and zero on ROUGE-L. The full-article output also repeats team names and misses the event details.

Scope and reproducibility

XSum reference wording frequently requires selecting an event and rephrasing it. TextRank and LexRank can preserve whole statements, yet centrality rewards repeated topics and may select background. A low-resource generator can shorten an article by copying a few salient words without recovering the main proposition. An extract can help by concentrating candidate words, or hurt by excluding a needed detail.

The test references had mean extractive density 1.02. A repeated trigram appeared in 500 of 600 full-article outputs and 492 of 600 hybrid outputs. Coverage did not prevent repetition in this small run. There is no coverage ablation here to estimate its effect.

The measured comparison changes the input and keeps neural weights fixed. It therefore characterizes inference on this checkpoint. It cannot isolate the effects of coverage, copy attention, model capacity, training duration, or XSum versus a high-overlap corpus. The subset, single seed, and absence of uncertainty intervals limit how much weight to put on small differences.

The measured run reduces embedding width, recurrent width, vocabulary, source length, and training set size. Training uses randomly initialized embeddings. No pretrained weights enter the experiment. The checkpoint, validation sweeps, per-article predictions and metric JSON enable inspection of the actual runs.

Reproducing the comparison

Run docker compose up --build from the extracted package, or install the pinned dependencies and run python code/run.py all. Set HF_HOME to a writable path if necessary. The Docker build pulls dependencies and the first experiment downloads the official Parquet data. The README also provides a smaller smoke command and independent task entry points.

Use task1.json to verify selected k and threshold, training.json to check the actual neural budget, and task3.json to compare the four matched systems. The pipeline prediction file includes the full article, selected extract, reference and neural outputs for each test ID.

References

[1] Narayan, S., Cohen, S. B., and Lapata, M. (2018). Don’t Give Me the Details, Just the Summary! Topic-Aware Convolutional Neural Networks for Extreme Summarization. EMNLP. https://aclanthology.org/D18-1206/

[2] Mihalcea, R., and Tarau, P. (2004). TextRank: Bringing Order into Text. EMNLP. https://aclanthology.org/W04-3252/

[3] Erkan, G., and Radev, D. R. (2004). LexRank: Graph-based Lexical Centrality as Salience in Text Summarization. JAIR 22, 457–479. https://doi.org/10.1613/jair.1523

[4] See, A., Liu, P. J., and Manning, C. D. (2017). Get To The Point: Summarization with Pointer-Generator Networks. ACL. https://aclanthology.org/P17-1099/

[5] EdinburghNLP. XSum dataset card and official split artifacts. https://huggingface.co/datasets/EdinburghNLP/xsum